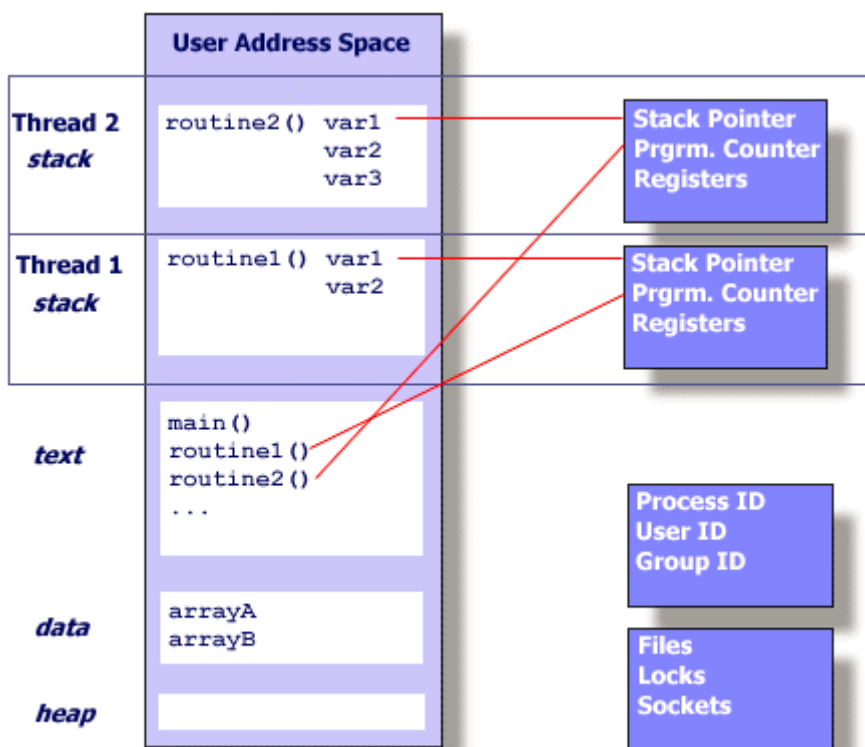


线程概念

线程概念

- 1、线程是进程的轻量级进程（light-weight process），每个线程都有一个PCB，创建线程和创建进程都一样的，子线程克隆了主线程的PCB。（线程共享 0-3G 用户空间）
- 2、在内核里看进程和线程一样的，都有各自不同的PCB，但PCB中指向内存资源的三级页表是相同的。
- 3、进程可以蜕变为线程。
- 4、线程本质是寄存器和栈
- 5、在linux下，线程最小的执行单位；进程是最小的分配资源单位。
- 6、CPU是以lwp为调度单位的。
- 7、查看lwp号：ps -Lf pid 和 ps -eLf
- 8、进程是分配资源的最小单元，线程是调度执行的最小单元。

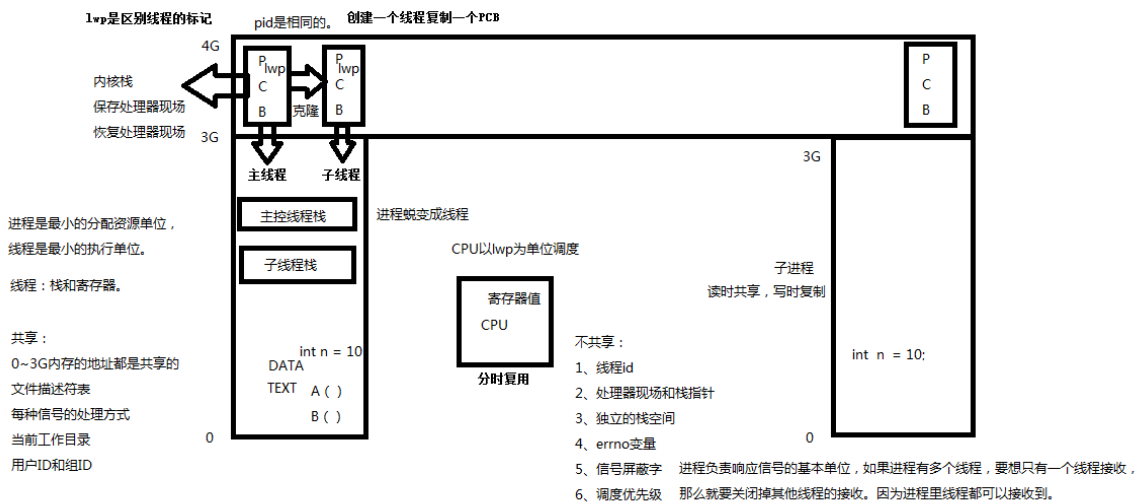


1、线程间共享的资源

- 1、文件描述符表：PCB里指向的一块内存；
- 2 每种信号的处理方式：

一、与进程相关的处理内容；

- 3、当前的工作目录；
- 4、用户ID和组ID；
- 5、内存地址空间，主线程和子线程都占用进程的0~3G空间。



2、线程间非共享资源

- 1、线程id，是在进程内有效，为线程之间方便通信。
- 2、处理器现场和栈指针（内核栈）
- 3、独立的栈空间（用户空间栈）
- 4、errno变量
- 5、信号屏蔽字：进程是接收信号的基本单位，进程里面的线程都可以接收到信号。但每个线程都可以设置信号的屏蔽，可以接收和不接收信号。（少用线程和信号混用）
- 6、调度优先级。

3、线程的优缺点

- 1、优点：
 - 1、提高程序的并发性；
 - 2、开销小，不用重新分配内存；

3、通信和共享数据方便。

2、缺点：

1、线程不稳定（库函数实现）；

2、线程调试比较困难（gdb支持不好）

3、线程无法使用unix经典事件，如信号。

4、查看线程pthread的函数

`man -k pthread` 查看与线程相关的函数

安装pthread相关man page : `sudo apt-get install manpages-posix manpages-posix-dev`

5、线程的原语

1、创建线程：

```
1 #include<pthread.h>
2
3
4 int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void*(*start_routine)
5
6
7 参数:
8 pthread_t *thread : 传递一个pthread_t变量地址进来, 用于保存新线程的tid(线程id).
9 const pthread_attr_t *attr : 线程属性设置, 如使用默认属性, 则传NULL
10 void*(*start_routine)(void*) : 函数指针, 指向新线程应该加载执行的函数模块。
11 void *arg : 指定线程将要加载调用的那个函数的参数。
```

返回值：成功返回0，失败返回错误号。由于pthread_create的错误码不保存在errno中，因此不能直接用perror()函数打印错误信息，可以先接收返回值，再用strerror(返回值)把错误码转换成错误码信息再打印。

创建线程并非原子操作。

编译时使用链接 `-lpthread`

`typedef unsigned long int pthread_t;`

1、pthread_create.c

```
1 #include <stdio.h>
2 #include <pthread.h>
3
4
5 pthread_t tid;
6
7
8 void *do_fun(void *arg)
9 {
10     int num = (int)arg;
11     //pthread_self()直接去当前线程PCB里获取线程id
12     //如果此时主控线程里的pthread_create还没调用返回，有可能tid此时还为0
13     //使用pthread_self函数重新获取，比较安全。
14     if(tid != pthread_self())
15     {
16         printf("hello");
17     }
18     while(1)
19     {
20         printf("child pid = %d  tid = %x arg = %d\n",getpid(),(unsigned int)pthread_s
21             sleep(1);
22     }
23 }
24
25
26 int main(void)
27 {
28     //1、创建线程，2、保存线程id到tid,3返回
29     pthread_create(&tid,NULL,do_fun,(void*)5);
30
31
32     while(1)
33     {
34         printf("main pid = %d  tid = %x\n",getpid(),(unsigned int)pthread_self());
35         sleep(1);
36     }
37     return 0;
38 }
```

6、pthread_self函数

```
1 pthread_t pthread_self(void); //获取调用线程tid
```

创建最大线程数：

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <pthread.h>
4
5
6 void *do_task(void *arg)
7 {
8     while(1)
9     {
10         printf("%x\n", (unsigned int)pthread_self());
11         sleep(1);
12     }
13     return NULL;
14 }
15
16
17 int main(void)
18 {
19     pthread_t tid;
20     int err, i = 1;
21
22
23     while(1)
24     {
25         err = pthread_create(&tid, NULL, do_task, NULL);
26         if(err != 0)
27         {
28             printf("err = %s \t %d\n", strerror(err), err);
29             break;
30         }
31         printf("thread : %d\n", ++i);
```

```

32     }
33     return 0;
34 }

```

```

thread : 7530
thread : 7531
thread : 7532
thread : 7533
err = 11      Resource temporarily unavailable

```

7、 pthread_exit函数

```

1 int pthread_exit(void *retval);并不是线程越多越好。
2
3
4 void *retval:线程退出时传递出的参数，可以是退出值或地址，如是地址时，不能是线程内部申请的局部变量。

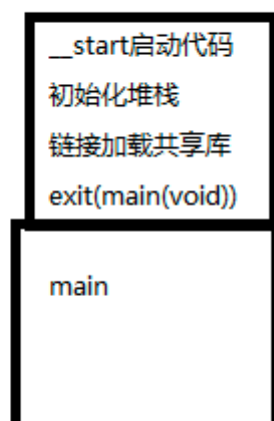
```

线程正常退出有三种方法：

- 1、 子线程正常退出时return;
- 2、 子线程使用pthread_exit退出;
- 3、 子线程被其他线程pthread_cancel取消。

任何线程里执行exit函数，都会导致整个进程终止。即使其他线程工作还没结束。

主控线程退出注意return或exit函数，确保子线程的工作完成后执行。



pthread_exit.c

```
1 #include <stdio.h>
2 #include <pthread.h>
3
4
5 struct STU
6 {
7     int id;
8     char str[128];
9 };
10
11
12 void *do_task(void *arg)
13 {
14     struct STU *s = (struct STU *)arg;
15     char name[256] = "yzmin";
16
17
18     printf("%p\t%s\n", name, name);
19     printf("%d\t%s\n", s->id, s->str);
20
21
22     return (void*)name;
23 }
24
25
26 int main(void)
27 {
28     pthread_t tid;
29     char *status;
30     struct STU s1 = {12, "qiuqiu"};
31
32
33     pthread_create(&tid, NULL, do_task, (void*)&s1); //创建子线程
34     pthread_join(tid, (void**)&status); //等待子线程结束，回收子线程
35
36
37     printf("status = %p\n", status);
38     printf("%s\n", status);
39
40
```

```
41     return 0;
42 }
```

```
xingwenpeng@ubuntu:~/LinuxCode/Pthread-08/kt$ ./a.out
0x7fdca919cdf0 yzmin
12      xiaoqiong
status = 0x7fdca919cdf0
xingwenpeng@ubuntu:~/LinuxCode/Pthread-08/kt$
```

注意：pthread_exit或者return返回的指针所指向的内存单元必须是全局的或者是用malloc分配的，不能在线程函数的栈上分配，因为当其他线程得到这个返回指针时，线程函数已经退出了。

8、pthread_cancel函数

```
1 int pthread_cancel(pthread_t thread); //在进程内某个线程可以取消另一个线程。
```

同一个进程的线程间，pthread_cancel向另一个线程发终止信号。系统并不会马上关闭被取消线程，只有在被取消线程下次系统调用时，才会真正结束线程。或调用pthread_testcancel，让内核去检测是否需要取消当前线程。

使用pthread_cancel被取消的线程，退出值。定义在Linux的pthread库中常数PTHREAD_CANCELED的值是-1。

```
#define PTHREAD_CANCELED ((void *) -1)
```

1、pthread_testcancel.c

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <string.h>
4
5
6 void *do_task(void *arg)
7 {
8     long long n = 0;
9     while(1)
10     {
11         n++;
12         pthread_testcancel(); //产生一个系统调用
```



```

13     ;
14 }
15 }
16
17
18 int main(void)
19 {
20     pthread_t tid;
21     char buf[1024];
22     pthread_create(&tid, NULL, do_task, NULL);
23     while(1)
24     {
25         fgets(buf, sizeof(buf), stdin);
26         if(strcmp(buf, "bye\n") == 0)
27         {
28             pthread_cancel(tid);
29             printf("bye child\n");
30         }
31         printf("main:%s\n", buf);
32     }
33     return 0;
34 }

```

2、pthread_cancel.c

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <pthread.h>
4 #include <unistd.h>
5
6
7 void *thr_fn1(void *arg)
8 {
9     printf("thread 1 returning\n");
10    return (void *)1;
11 }
12
13
14 void *thr_fn2(void *arg)
15 {
16    printf("thread 2 exiting\n");

```

```
17     pthread_exit((void *)2);
18 }
19
20
21 void *thr_fn3(void *arg)
22 {
23     while(1)
24     {
25         printf("thread 3 writing\n");
26         sleep(1);
27     }
28 }
29
30
31 int main(void)
32 {
33     pthread_t tid;
34     void **tret;
35
36
37     pthread_create(&tid, NULL, thr_fn1, NULL);
38     pthread_join(tid, &tret);
39
40
41     printf("thread 1 exit code %d\n", (int)tret);
42
43
44     pthread_create(&tid, NULL, thr_fn2, NULL);
45
46
47     pthread_join(tid, &tret);
48     printf("thread 2 exit code %d\n", (int)tret);
49     pthread_create(&tid, NULL, thr_fn3, NULL);
50
51
52     sleep(3);
53     pthread_cancel(tid); //取消 返回值 为 -1.
54     pthread_join(tid, &tret);
55     printf("thread 3 exit code %d\n", (int)tret);
56
57
58     return 0;
59 }
```

9、pthread_join函数

```
1 int pthread_join(pthread_t thread, void **retval);
2
3
4 pthread_t thread: 回收线程的tid
5
6
7 void **retval:接收退出线程传出的返回值
```

返回值：成功返回0，失败返回错误号。

阻塞函数，主线程会挂起等待。

- 1) 如果thread线程通过return返回，retval所指向的单元里,存放的是thread线程函数的返回值。
- 2) 如果thread线程被别的线程调用pthread_cancel异常终止掉，retval所指向的单元里,存放的是常数PTHREAD_CANCELED。
- 3) 如果thread线程是自己调用pthread_exit终止的，retval所指向的单元,存放的是传给pthread_exit的参数。
- 4) 如果对thread线程的终止状态不感兴趣，可以传NULL给retval参数。

10、pthread_detach线程分离函数

```
1 pthread_detach(pthread_t tid); //分离线程tid
2
3
4 参数:
5 pthread_t tid:分离线程tid号
```

返回值：成功返回0，失败返回错误号。

线程分离后，线程一旦终止，系统立刻回收它占用的所有资源，不需要主控线程回收。

当线程处于detach状态（游离态线程），再调用pthread_join，将返回EINVAL。

线程的归宿方式：

- 1、被主控线程pthread_join回收；
- 2、设置成游离态，线程结束时系统自动回收线程的资源。

1、pthread_detach.c

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <pthread.h>
4
5
6 void *do_task(void *arg)
7 {
8     int n = 5;
9     while(n-->0)
10     {
11         printf("%x\n",pthread_self());
12         sleep(1);
13     }
14
15
16     return (void*)3;
17 }
18 //线程的归属二选一
19 //1、被主控线程pthread_join回收
20
21 //2、设置成游离态，线程结束时系统自动回收线程的资源
22 int main(void)
23 {
24     pthread_t tid;
25     int err,*status;
26
27
28     pthread_create(&tid,NULL,do_task,NULL);
29     //sleep(5);
30     pthread_detach(tid);//设置子线程为分离态
```

```

31 //分离态由系统去回收子线程
32 while(1)
33 {
34     err = pthread_join(tid,(void*)&status);
35     if(err!=0)
36     {
37         printf("err= %s\n",strerror(err));
38         break;
39     }
40     else
41     {
42         printf("child exit %d\n",(int)status);
43     }
44 }
45 return 0; //主控线程退出了，那么这个进程就结束了。
46 }

```

11、pthread_equal函数

```

1 int pthread_equal(pthread_t t1, pthread_t t2); //判断线程号是否相等。

```

比较两个线程是否相等

12、线程终止方式

如果需要只终止某个线程而不终止整个进程，有三种方法：

- 1、从线程主函数return。这种方法对主控线程不适用，从main函数return相当于调用exit。
- 2、一个线程可以调用pthread_cancel终止同一个进程中的另一个线程。
- 3、线程可以调用pthread_exit终止自己。

线程属性

1、线程属性的结构体：pthread_attr_t

```

1 //
2 typedef struct
3 {
4     int detachstate; //线程的分离状态
5     int schedpolicy; //线程调度策略
6     struct sched_param schedparam; //线程的调度参数
7     int inheritsched; //线程的继承性
8     int scope; //线程的作用域
9     size_t guardsize; //线程栈末尾的警戒缓冲区大小
10    int stackaddr_set; //线程的栈设置
11    void* stackaddr; //线程栈的位置
12    size_t stacksize; //线程栈的大小
13 }pthread_attr_t;

```

属性值不能直接设置，须使用相关函数进行操作。

默认的属性为非绑定、非分离、缺省M的堆栈，与父进程同样级别的优先级。

2、线程属性初始化

```

1 int pthread_attr_init(pthread_attr_t *attr); //attr传出参数

```

先初始化线程属性，再pthread_create创建线程。之后须用pthread_attr_destroy函数来释放资源。

3、销毁线程属性所占用的资源

```

1 int pthread_attr_destroy(pthread_attr_t *attr);

```

4、线程的分离状态：

1、设置线程属性

```
1 int pthread_attr_setdetachstate(pthread_attr_t *attr,int detachstate);//设置线程属性,
```

detachstate : 传入参数

2、 获取线程属性

```
1 int pthread_attr_getdetachstate(pthread_attr_t *attr,int *detachstate);//获得线程属性,
2
3
4 pthread_attr_t *attr : 被已初始化的线程属性
5
6
7 int *detachstate (传出参数) : PTHREAD_CREATE_DETACHEN分离线程
8
9
10                                PTHREAD_CREATE_JOINABLE非分离线程
```

3、线程的分离状态决定一个线程以什么样的方式来终止自己。

1、非分离状态

线程的默认属性是非分离状态，这种情况下，原有的线程等待创建的线程结束。只有当pthread_join()函数返回时，创建的线程才算终止，才能释放自己占用的系统资源。

2、分离状态

分离线程没有被其他的线程所等待，自己运行结束了，线程也就终止了，马上释放系统资源。

pthread_attr_detach.c

```
1 #include <stdio.h>
2 #include <string.h>
```

```

3 #include <pthread.h>
4
5
6 void *do_task(void *arg)
7 {
8     int m = 20;
9     //pthread_detach(pthread_self());//第一种线程回收方式
10    while(m--)
11    {
12        printf("%x\n",(unsigned int)pthread_self());
13        sleep(1);
14    }
15    return NULL;
16 }
17
18
19 int main(void)
20 {
21     pthread_t tid;
22     int err,i = 1,n = 100;
23     //第二种，在线程创建之前，使用线程属性设置
24     pthread_attr_t attr;
25     pthread_attr_init(&attr);//默认初始化属性
26     pthread_attr_setdetachstate(&attr,PTHREAD_CREATE_DETACHED);//单独设置游离态
27
28
29     while(n--)
30     {
31         err = pthread_create(&tid,&attr,do_task,(void*)n);
32         if(err != 0)
33         {
34             printf("err = %d \t %s\n",err,strerror(err));
35             break;
36         }
37         printf("thread : %d\n",++i);
38     }
39     //避免进程退出
40     while(1)
41     {
42         printf("main thread\n");
43         sleep(1);
44     }
45     return 0;
46 }

```


5、线程的栈地址（stack address）

```
1 int pthread_attr_setstackaddr(pthread_attr_t *attr, void *stackaddr);
2
3
4 int pthread_attr_getstackaddr(pthread_attr_t *attr, void **stackaddr);
5
6
7 attr: 指向一个线程属性的指针
8 stackaddr: 返回获取的栈地址
9 返回值: 若是成功返回0, 否则返回错误的编号
10 说明: 函数已过时, 一般用下面讲到的pthread_attr_getstack来代替
```

当进程栈地址空间不够用时, 指定新建线程使用由malloc分配的空间作为自己的栈空间。

6、线程的栈大小（stack size）

设置线程属性： 栈的大小。

```
1 #include <pthread.h>
2
3
4 int pthread_attr_setstacksize(pthread_attr_t *attr, size_t stacksize);
5
6
7 int pthread_attr_getstacksize(pthread_attr_t *attr, size_t *stacksize);
8
9
10 attr 指向一个线程属性的指针
11
12
13 stacksize 返回线程的堆栈大小
14
15
```

16 返回值：若是成功返回0, 否则返回错误的编号
17 以用不多。

整合了：栈地址和栈大小

```
1 int pthread_attr_setstack(pthread_attr_t *attr, void *stackaddr, size_t stacksize);
2
3
4 int pthread_attr_getstack(pthread_attr_t *attr, void **stackaddr, size_t *stacksize);
5
6
7 attr 指向一个线程属性的指针
8
9
10 stackaddr 返回获取的栈地址
11
12
13 stacksize 返回获取的栈大小
14
15
16 返回值：若是成功返回0, 否则返回错误的编号
```

pthread_attr_detach2.c

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <pthread.h>
4
5
6 void *do_task(void *arg)
7 {
8     int m = 20;
9     //方案一：pthread_detach(pthread_self());
10    while(m-->0)
11    {
12        //printf("%x\n", (unsigned int)pthread_self());
13        sleep(1);
14    }
15}
```

```
13     sleep(1);
14 }
15 return NULL;
16 }
17
18
19 int main(void)
20 {
21     pthread_t tid;
22     int err,i = 1,n = 100;
23     //方案2: 在创建线程之前, 构造线程属性
24     pthread_attr_t attr;
25     pthread_attr_init(&attr);//默认属性
26     pthread_attr_setdetachstate(&attr,PTHREAD_CREATE_DETACHED);//设置线程的游离态
27
28
29     if((pthread_attr_setstacksize(&attr,1024*1024*2)==0))
30     {
31         //printf("ok\n");
32         //sleep(2);
33     }
34
35
36     while(1)
37     {
38         err = pthread_create(&tid,&attr,do_task,NULL);
39
40
41         if(err != 0)
42         {
43             printf("err = %s \t %d\n",strerror(err),err);
44             break;
45         }
46         printf("thread : %d\n",++i);
47     }
48
49
50     while(1)
51     {
52         printf("main thread\n");
53         sleep(1);
54     }
55
56
```

```
57 |     return 0;
58 | }
```

7、NPTL

NPTL实现机制（POSIX）,Native POSIX Thread Library

查看当前pthread库版本： `getconf GUN_LIBPTHREAD_VERSION`

使用线程库时 gcc 指定 `-lpthread`

细节注意

1、主线程退出其他线程不退出，主线程应调用 `pthread_exit`

2、避免僵线程：

`pthread_join`

`pthread_detach`

`pthread_create`指定分离属性

被 `join` 线程可能在 `join` 函数返回前就释放完自己的所有内存资源，所以不应当返回被回收线程栈中值。

3、`malloc`和`mmap`申请的内存可以被其他线程释放

4、应避免在多线程模型中调用 `fork` 除非，马上 `exec`, 子进程中只有调用 `fork` 的线程存在，其他线程在子进程中均 `pthread_exit`

5、信号的复杂语义很难和多线程共存，应避免在多线程引入信号机制。

6、如果线程终止时没有释放加锁的互斥量，则改互斥量不能再被使用。

